1. Práctica 5: Configuración de Timers

En esta práctica vamos a introducir el concepto de temporizador. Son unos periféricos del procesador que permiten, cada cierto tiempo, generar interrupciones y, con ello, disparar funcionalidades concretas que hayamos programado. En este caso programaremos uno para hacer parpadear un LED, y otro diferente para utilizar como *debouncer* del botón.

1.1. Objetivos

- Entender el funcionamiento de los temporizadores internos del sistema.
- Interconectar interrupciones de varios periféricos para lograr funcionalidades avanzadas.

1.2. Fundamentos teóricos

Un temporizador o *timer* en un microcontrolador es un periférico hardware que, en esencia, es un contador digital. La cuenta se realiza en función de una señal de reloj de entrada, y al llegar a ciertos valores (configurables) dispara interrupciones.

Es capaz de medir intervalos de tiempo de forma independiente a la ejecución de código en la CPU, liberándola pues para que haga otras tareas en lugar de esperas activas a ciertos eventos. Los temporizadores suelen contar con varios registros para su control, entre otros:

- `PSC` o *prescaler*: Un registro que sirve para dividir la frecuencia de entrada. Normalmente, la frecuencia del procesador están en el orden de MHz. El prescaler suele dividirlo al orden de KHz o incluso menos, según las necesidades de temporización.
- `CNT` o *counter*: El registro que lleva la cuenta actual del timer. Suma 1 cada vez que la señal de reloj, dividida entre el factor del prescaler, se activa. Su frecuencia de actualización f_{CNT} es por tanto:

$$f_{CNT} = \frac{f_{CLK}}{\texttt{PSC} + 1}$$

- `ARR` o *auto-reload*: Valor máximo de conteo. Cuando `CNT` alcanza este valor, se resetea y genera un evento de interrupción que podremos controlar. El periodo de evento T_{UEV} es por tanto:

$$T_{UEV} = \frac{\texttt{ARR} + 1}{f_{CNT}} = \frac{(\texttt{PSC} + 1)(\texttt{ARR} + 1)}{f_{CLK}}$$

- `CCR` o *capture compare*: Registros que permiten diversas configuraciones de comparación con CNT (mayor que, menor, igual ...) para medir la duración de señales externas o generar formas de onda PWM, entre otras funcionalidades. En ocasiones también permiten interupciones adicionales a la de `ARR`.

Podemos ver un diagrama de funcionamiento en la Figura 1.

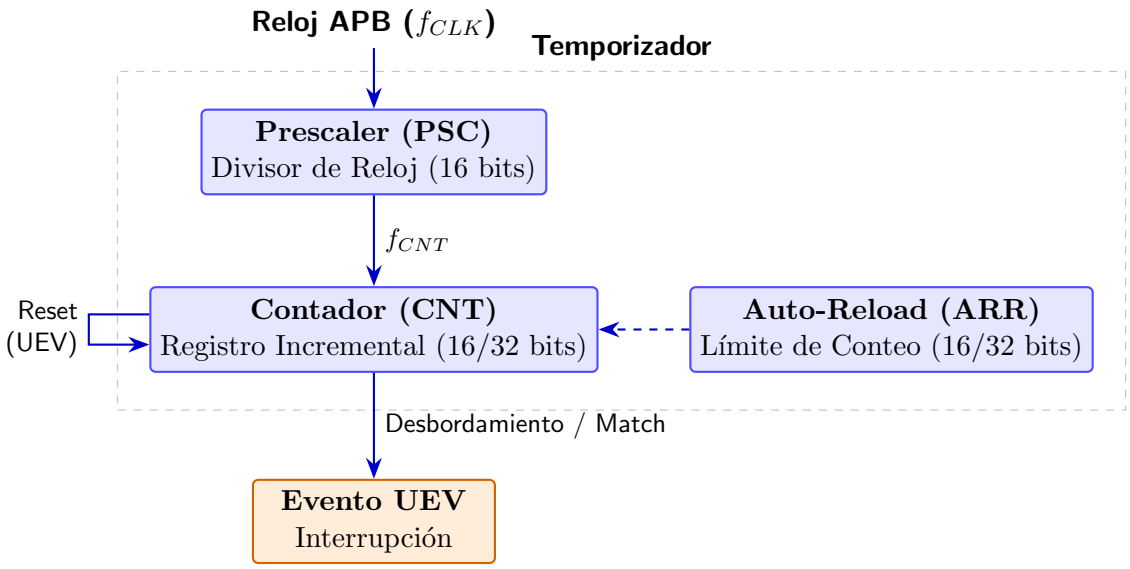


Figura 1: Diagrama de funcionamiento simplificado para los timers

1.3. Desarrollo de la práctica

Lo primero, como siempre, es ir añadiendo a nuestro `bsp.h` lo necesario para poder gestionar los timers. Vamos a utilizar dos, `TIM2` y `TIM3`. Consulta `rm0431 1.6` para ver las direcciones base de memoria, y `rm0431 19.4` para los registros de los temporizadores que permitan crear el `typedef` adecuado.

```
1  //... completar
2  #define TIM2_BASE_ADDR
3  #define TIM3_BASE_ADDR
4
5  //Structure for Timers (TIM2 / TIM3)
6  typedef struct
7  {
8      volatile uint32_t CR1;        /*!< TIM control register 1,
           Address offset: 0x00 */
9      volatile uint32_t CR2;        /*!< TIM control register 2,
           Address offset: 0x04 */
10     //... completar
11 } TIM_TypeDef;
12
13 #define TIM2                ((TIM_TypeDef *) TIM2_BASE_ADDR)
14 #define TIM3                ((TIM_TypeDef *) TIM3_BASE_ADDR)
```

A continuación tenemos que crear las funciones necesarias para el control de los timers:

```
1  //funciones de configuración
2  //... activa en CR1 el bit correspondiente para activar el contador
3  static inline void TIM_ENABLE_COUNTER(volatile TIM_TypeDef *TIMx, uint32_t
       enable);
4  //... activa el modo oneshot desde CR1
5  static inline void TIM_ENABLE_ONESHOT(volatile TIM_TypeDef *TIMx, uint32_t
       enable);
6  //... configura el registro PSC con el valor de preescalado
7  static inline void TIM_SET_PRESCALER(volatile TIM_TypeDef *TIMx, uint32_t value)
       ;
8  //... configura el registro ARR con el valor de autoreload
9  static inline void TIM_SET_AUTO_RELOAD(volatile TIM_TypeDef *TIMx, uint32_t
```

```
    value);
//... activa la interrupción UIE en el registro DIER
static inline void TIM_ENABLE_INT(volatile TIM_TypeDef *TIMx);
//funciones de uso
//... borra el flag pendiente del registro SR (correspondiente a UIF)
static inline void TIM_CLEAR_FLAG(volatile TIM_TypeDef *TIMx);
//... comprueba si hay flag pendiente en el registro SR
static inline uint32_t TIM_CHECK_FLAG(volatile TIM_TypeDef *TIMx);
```

También es necesario crear una función que nos permita inicializar el timer, llamando a las anteriores de configuración:

```
//congiura prescaler, autoreload, activa counter y oneshot si es necesario y
    activa interrupciones
static void TIM_INIT(TIM_TypeDef *TIMx, uint32_t preescaler, uint32_t autoreload
    , uint32_t enable_counter, uint32_t enable_oneshot);
```

Y por último tendremos que activar los bits correspondientes en `APB1ENR` (`rm0431 5.3.13`), y adicionalmente activar en `NVIC` (`rm0431 9.1.2`) las líneas de interrupción adecuadas:

```
RCC_APB1ENR->bits.TIM2EN = 1;
RCC_APB1ENR->bits.TIM3EN = 1;
TIM_INIT(TIM2, 15999, 500, 1, 0); //500 milisegundos, activo
TIM_INIT(TIM3, 15999,  10, 0, 1); //10 milisegundos, inactivo, oneshot
//Enable interrupts
NVIC_ENABLE_INT(TIM2_IRQn);
NVIC_ENABLE_INT(TIM3_IRQn);
```

Ahora podemos, por fin, hacer uso de las funciones de los timers para ver su efecto:

```
// Handles Task 1: Blink LED1 (Green) each 0.5 seconds
void TIM2_IRQHandler(void) {
    //... completar comprobar que el flag está activo
    if () {
        //... completar borrar el flag
        //... completar funcionalidad deseada
    }
}
```

La funcionalidad que queremos se muestra en la Figura 2

- `TIM2` enciende o apaga el led verde (PB1) cada vez que se dispara. Para ello, asegúrate de haber activado dicho puerto y pin como `GPIO`.
- `TIM3` sirve como *debouncer* para el botón (en PA0). Para ello debemos conseguir que, al pulsarse el botón, se inhabilite durante el tiempo que tarde el timer en dispararse. Si tras ese tiempo el botón sigue pulsado, contaremos que es una pulsación de verdad. En caso contrario, lo omitiremos. En ambos casos, al terminar el temporizador volveremos a activar las interrupciones del botón. Como está en modo *oneshot*, cada vez que lo activemos se disparará solo una vez, evitando problemas.

Para el botón, debemos comprobar tras la activación que el bit correspondiente de `PR` está activo en `EXTI` (`rm0431 10.9.6`). Si es así, lo borramos y a continuación desactivamos la interrupción desde el registro `IMR` (`rm0431 10.9.1`). Finalmente activamos el temporizador, reseteando `CNT` a cero y activándolo desde `CR1` (`rm0431 19.4.1`).

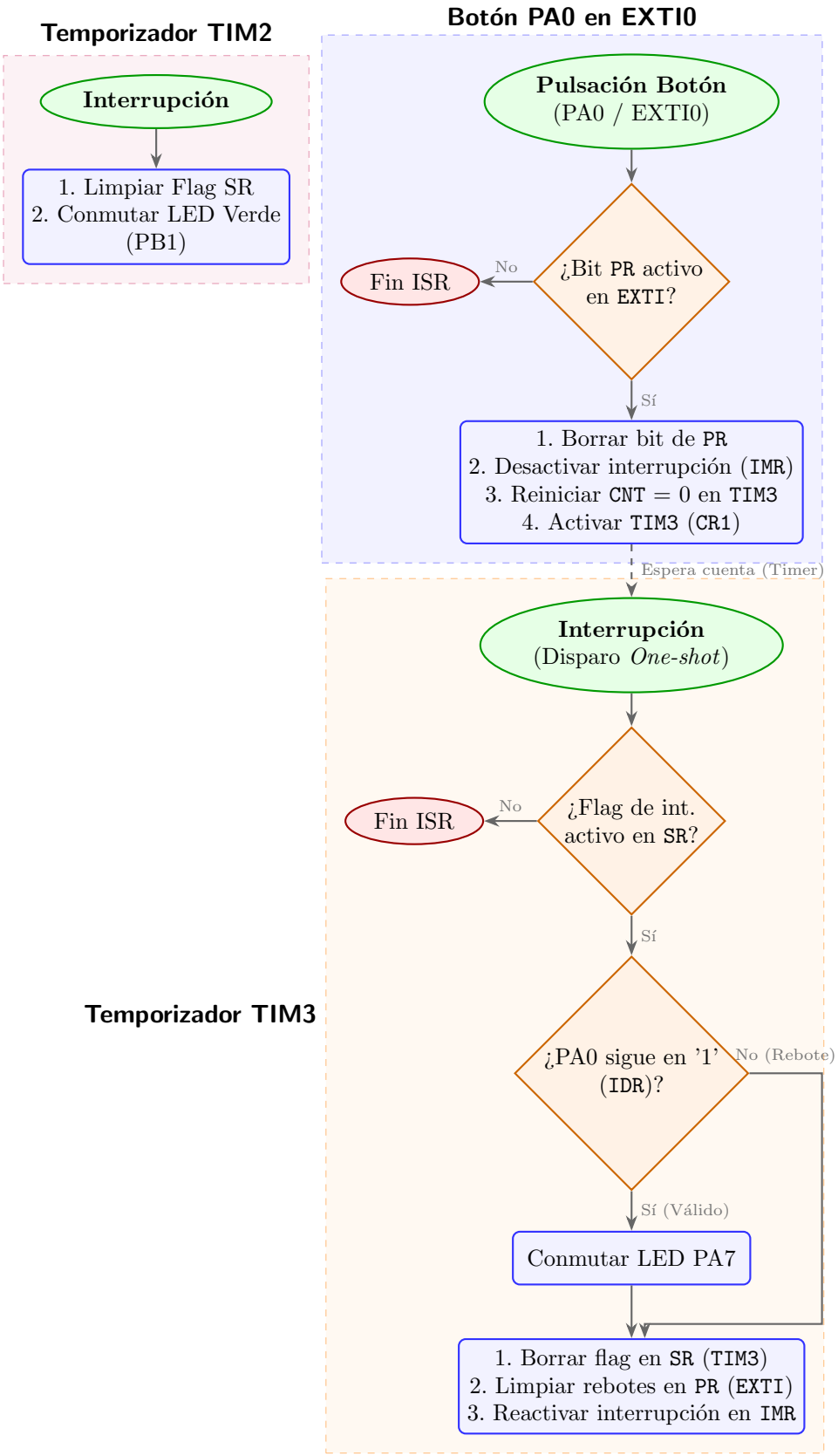


Figura 2: Diagrama de flujo funcional de las interrupciones.

```
void EXTI0_IRQHandler(void)
{
    //... comprobar si la interrupción está pendiente
    if () {
        //... borrar bit pendiente
        //... desactivar interrupciones de botón
        //... resetear cuenta de timer
        //... activar timer
    }
}
```

En cuanto al timer 3, debemos en primer lugar comprobar y borrar si es necesario el flag de interrupción desde `SR` (`rm0431 19.4.5`). A continuación, y si el botón sigue pulsado (podemos comprobarlo desde el registro `IDR` del `GPIO` (`rm0431 6.4.5`)), efectuar la función deseada (por ejemplo hacer parpadear el LED conectado a PA7). Tras ello, borraremos interrpciones pendientes del botón (por si se hubiera quedado pillado con rebotes) desde `PR` de `EXTI` (`rm0431 10.9.6`), y re-activaremos las interrupciones de botón en `IMR` (`rm0431 10.9.1`). Una cuestión importante es que, gracias a haber configurado el temporizador en modo *oneshot*, no es necesario desactivar sus propias interrupciones ya que no volverá a dispararse hasta reiniciar la cuenta.

```
// Debounce Lockout Timer Handler (Fires 10ms after initial edge)
void TIM3_IRQHandler(void) {
    //... comprobar flag de interrupción
    if () {
        //... borrar flag de interrpción
        //... verificar que sigue pulsado PA0
        if () {
            //... funcionalidad... Toggle led de PA7
        }
        //... borrar interrpciones pendientes para EXTI0 (botón)
        //... reactivar interrupciones para EXTI0
    }
}
```

1.4. Para hacer más

Aunque hay varios temporizadores (14 en concreto en el caso de **STM32F723IE**(`rm0431 18,19,20`)) para sistemas complejos con necesidad de medir una multitud de intervalos, pueden quedarse cortos. Para solventar este problema, cada temporizador es capaz de generar interrupciones en varios modos diferentes y con varios contadores distintos. Como todas estas interrupciones comparten ciertos aspectos de configuración, no podrá haber demasiada diferencia entre los tiempos que midan, aún así da mucha versatilidad. Investiga cómo puedes hacer que haya varias interrupciones diferentes desde un mismo temporizador (`rm0431 19`). (**NOTA:**) las interrupciones vendrán por la misma línea, y habrá que averiguar, programáticamente, qué parte del temporizador las causó (`rm0431 19.4.4,19.4.5`).